

# Proxia: Unified AI Gateway - Final Project Report

## 1. Project Overview

github repo

<https://github.com/leepokai/proxia>

### Background & Motivation

在當前的 LLM 應用開發中，開發者面臨著嚴峻的「The Fragmentation Nightmare」。雖然 AI 模型百家爭鳴，但缺乏統一標準導致了以下三大核心痛點：

#### API Chaos：

- 不同供應商的請求與回應格式完全不統一。OpenAI 的 Chat API 使用標準的 `messages` 陣列結構；然而，Anthropic (Claude) 則採用 `system` 提示詞搭配 `max_tokens` 的參數設計
- 這種結構上的差異意味著開發者必須在程式碼中撰寫大量的 `if-else` 邏輯來處理不同廠商的 JSON 格式，增加了程式碼的複雜度。

#### SDK Hell

- 為了在應用程式中整合多個模型，開發者往往被迫引入並管理多個臃腫的官方 SDK
- 僅僅是為了發送簡單的 HTTP 請求，卻需要維護多套依賴庫 (Dependencies)，這不僅增加了專案的體積，也提高了依賴衝突的風險。

#### High Maintenance Cost

- 當業務需求變更或新模型（如 Claude 3.5 或 GPT-4o）發布時，切換模型往往不只是更改設定檔那麼簡單，而是需要重寫底層的呼叫邏輯與錯誤處理機制
- 既有的錯誤處理 (Error Handling) 邏輯在不同廠商間無法共用，導致系統穩定性難以統一保障。

---

## The Solution

為了徹底解決上述問題，Proxia 提出了一個「Write Once, Call Any Model」的統一介面架構

## Single Endpoint

- Proxia 提供單一的 API 入口 `POST /v1/chat`，該介面在格式上與 OpenAI API 100% 相容
- 後端核心負責處理所有針對 OpenAI、Anthropic 及 Gemini 等不同供應商的請求轉發與協定轉換

## Decoupling Architecture

- 透過 Proxia 的中介層設計，應用程式邏輯與底層 AI 供應商徹底解耦。開發者只需依照 OpenAI 的標準格式撰寫一次程式碼，即可隨意切換底層使用的模型，無需修改客戶端程式碼

---

## Feature List

### Dynamic Routing

- Proxia 的路由器會解析請求 Body 中的 `model` 欄位。系統根據模型名稱的前綴 (Prefix) 自動決定路由目的地，無需額外的設定檔更改
  - **Anthropic 路由**：若模型名稱以 `claude*` 開頭，請求將自動轉發至 Anthropic API
  - **OpenAI 路由**：若模型名稱以 `gpt*` 或 `o1*` 開頭，請求將路由至 OpenAI API
- **強制指定 (Override)**：針對邊緣案例，開發者亦可透過自訂 `provider` 欄位強制指定路由路徑

### Response Normalization

- **統一輸出格式**：不同上游的回應格式差異巨大。例如，Claude 的原始回應是扁平結構且欄位命名不同（如 `{ content: [{ text: "Hi" }] }`）
- **Adapter**：Proxia 實作了回應適配器，將來自 Anthropic 等非標準來源的回應，強制包裝 (Wrap) 成 OpenAI 標準的 `{ choices: [{ message: { content: ... } }] }` 格式。這確保了客戶端接收到的資料結構始終一致。

### Flexible Auth & Rate Limiting

- **雙重驗證模式**：

1. **Gateway Keys**：由 Proxia 簽發的金鑰。
2. **Upstream Keys (BYOK)**：支援「自帶金鑰」(Bring Your Own Key) 模式。使用者可直接傳入 OpenAI 或 Anthropic 的原生金鑰，Proxia 會以 Zero-overhead 的方式透傳至上游，不進行額外限流。

## Key Management Dashboard

- **現代化技術棧**：前端採用 Next.js 14 (App Router) 構建，搭配 Supabase (PostgreSQL) 處理資料持久化與金鑰管理。
- **視覺化功能**：提供完整的 Web Dashboard，使用者可在此管理 Gateway Keys、查看使用量，並透過內建的 Playground 直接測試不同模型的路由與回應效果。

## 2. Technical Architecture

本專案採用 Go 語言 (1.22+) 開發核心後端，以前端 Next.js 14 搭配 Supabase 進行管理。系統設計遵循 Clean Architecture 原則，確保高內聚低耦合。

### 2.1 System Architecture

系統請求處理流程如下：請求首先通過 Middleware 鏈 (CORS, Logging, Auth)，接著進入 Router 進行 payload 解析與 Provider 選擇，最後由具體的 Handler 轉發至上游 API。

```
graph TD
    Client([Client / SDK]) -->|POST /v1/chat| LoadBalancer[Load Balancer / Docker Port]

    subgraph "Proxia Backend (Go 1.22)"
        LoadBalancer --> Middleware
    end

    subgraph "Middleware Chain"
        Middleware[Middleware Layer]
        Auth[Auth & Rate Limit]
        Log[Logging & CORS]
    end
    end

    Middleware -->|Valid Token| Router[Router: handleChat]

    subgraph "Dynamic Routing Logic"
        Router -->|Parse Model| Selector{Select Provider}
    end
```

```

Selector → |prefix: gpt/o1| H_OpenAI[OpenAI Handler]
Selector → |prefix: claude| H_Claude[Claude Handler]
Selector → |Other| H_Default[Default Handler]
end

H_OpenAI → |Interface: SendRequest| Adapter[Response Normalizer]
H_Claude → |Interface: SendRequest| Adapter
end

H_OpenAI ↔ |HTTP/JSON| API_OpenAI[OpenAI API]
H_Claude ↔ |HTTP/JSON| API_Anthropic[Anthropic API]

Adapter → |Standardized JSON| Client

```

## 2.2 Go Language Features

### A. Concurrency & Context Management

我們充分利用 Go 的 Goroutine 模型處理高併發請求。每個 HTTP 請求皆在獨立的 Goroutine 中執行。為了防止上游 API 回應過慢導致 Goroutine Leaks，我們在 handleChat 中強制實作了 Context Timeout 機制：

```

// handleChat: 強制設定 60 秒超時控制，避免上游無回應導致連線堆積

ctx, cancel := context.WithTimeout(r.Context(), 60*time.Second)defer cancel()// 將帶有 Timeout 的 Context 傳遞給 Provider

respBytes, err := provider.SendRequest(ctx, cleanPayload, upstreamKey)

```

### B. Interfaces & Decoupling

為了實現 "Write Once, Call Any Model"，我們定義了統一的 AIProvider 介面。Router 不需要知道具體的實作細節，僅需依賴介面進行操作。這使得未來擴充 Gemini 或 Mistral 等新模型時，無需修改核心路由邏輯。

Dynamic Dispatch:

我們利用 Go 的 map 與字串處理功能，實現了高效的模型分派策略：

```
// selectProvider 根據模型名稱前綴自動決定路由目的地
func (a *App) selectProvider(model string) string {
    m := strings.ToLower(strings.TrimSpace(model))
    // 自動路由 Claude 系列模型
    if strings.HasPrefix(m, "claude") || strings.Contains(m, "anthropic") {
        if _, ok := a.providers["claude"]; ok {
            return "claude"
        }
    }

    // 自動路由 GPT 與 O1 系列模型
    if strings.HasPrefix(m, "gpt") || strings.Contains(m, "openai") || strings.HasPrefix(m, "o1") {
        if _, ok := a.providers["openai"]; ok {
            return "openai"
        }
    }

    // Fallback 至預設供應商
    return a.defaultProvider
}
```

### C. Middleware Design (中介軟體設計)

我們採用了 Go 標準的 `http.Handler` 裝飾器模式 (Decorator Pattern) 來堆疊 Middleware。這種設計讓我們能彈性地組合 CORS、Logging 與 Authentication 功能，且保持程式碼整潔。

```
// 透過函數閉包 (Closure) 組合 Middleware 鏈 // 執行順序: CORS → Logging -
> Auth → RateLimit → Handler
secured := auth.Middleware(a.store, a.limiter, a.logger)
chatHandler := corsMiddleware(loggingMiddleware(a.logger, secured(http.
HandlerFunc(a.handleChat))))
a.mux.Handle("POST /v1/chat", chatHandler)
```

### D. Error Handling (錯誤處理)

針對上游供應商可能回傳的各種錯誤格式（如 OpenAI 與 Anthropic 的錯誤 JSON 結構不同），我們實作了統一的錯誤萃取邏輯 `extractProviderMessage`，確保客戶端收

到的一律是標準化的 HTTP Status 與錯誤訊息。

```
// 統一處理上游錯誤，嘗試解析不同廠商的 JSON Error 結構
if hse, ok := err.(*handlers.HTTPStatusError); ok {
    msg := extractProviderMessage(hse.Body)
    a.logger.Warnf("upstream provider error (%d): %s", hse.StatusCode, msg)
    utils.WriteError(w, hse.StatusCode, "provider error: "+msg)
    return
}
```

## 2.3 核心功能實作：回應適配器 (Response Adapter)

為了達成 "Write Once, Call Any Model"，我們必須解決不同 AI 供應商回傳格式不一致的問題。Proxia 實作了一個輕量級的 JSON 正規化引擎，確保客戶端收到的資料結構始終符合 OpenAI 標準。

### 正規化流程圖 (Normalization Flow):

flowchart LR

```
RawJSON["原始 JSON (Provider A/B)"] → Unmarshal{"解析結構"}
Unmarshal → Inject["注入欄位: provider"]
Inject → CheckID{"檢查 ID?"}
CheckID -- 無 → GenID["生成 ShortID"]
CheckID -- 有 → CheckTime{"檢查 Created?"}
CheckTime -- 無 → FillTime["填補當前 Unix Timestamp"]
CheckTime -- 有 → Marshal["封裝為標準 JSON"]
Marshal → Client
```

關鍵程式碼實作 (Go Implementation): 我們利用 Go 的 `map[string]any` 進行動態 JSON 操作，而非定義僵化的 Struct，這賦予了系統極高的彈性來處理未知的欄位。

```
// NormalizeProviderResponse: 將任意上游回應轉換為統一格式
func NormalizeProviderResponse(provider string, raw []byte) ([]byte, error) {
    var m map[string]any
    // 1. 動態解析原始 JSON
    if err := json.Unmarshal(raw, &m); err != nil {
        return nil, err
    }
}
```

```

}

// 2. 注入識別資訊 (Metadata Injection)
m["provider"] = provider

// 3. 填補缺失欄位 (Polyfill missing fields)
// 某些模型 (如 Local LLM) 可能不回傳 ID 或 Created 時間，此處自動補齊
if _, ok := m["id"]; !ok {
    m["id"] = "gateway-" + shortID()
}
if _, ok := m["created"]; !ok {
    m["created"] = time.Now().Unix()
}

// 4. 重新封裝
return json.Marshal(m)
}

```

## 測試與品質保證 (Testing & QA)

### 單元測試 (Unit Testing)

本專案已針對核心工具模組撰寫完整的單元測試，測試檔案位於 `backend/utils/tests/`，並採用 獨立的 `utils_test` 套件，確保測試僅透過公開 API 進行，避免與內部實作細節耦合，提升測試的可靠性與可維護性。

加密模組測試 (`crypto_test.go`)

#### 1. TestDerive32Key：金鑰派生邏輯測試

此測試覆蓋 `Derive32Key` 在不同輸入條件下的行為，確保金鑰產生邏輯安全且可預期：

- Base64 編碼金鑰
- 輸入：Base64 編碼的 32 位元組資料
- 驗證：可正確解碼並回傳原始 32-byte 金鑰
- 非 Base64 輸入
- 輸入：任意非 Base64 字串
- 驗證：系統正確回退 (fallback) 至 SHA-256 雜湊產生金鑰
- 空輸入錯誤處理

- 輸入：空字串
- 驗證：回傳 missing encryption key 錯誤

此測試確保系統在各種設定情境下都能產生穩定且安全的加密金鑰。

#### 1. TestEncryptDecryptString：加密／解密往返測試

- 使用 AES-256-GCM 進行字串加密
- 驗證加密後的密文可成功解密回原始明文
- 確保加密輸出不為空，避免無效加密結果

此測試確保對稱式加密流程的正確性與完整性。

#### 1. TestDecryptStringErrors：解密錯誤處理測試

測試解密失敗時的防禦行為：

- 無效 Base64 密文
- 輸入：格式錯誤的 Base64 字串
- 驗證：回傳 invalid encrypted payload 錯誤
- 錯誤金鑰解密
- 使用金鑰 A 加密、金鑰 B 嘗試解密
- 驗證：回傳 decrypt failed 錯誤

此測試確保系統在錯誤或攻擊情境下能正確拒絕不合法請求。

HTTP 回應與正規化測試 (response\_test.go)

#### 1. TestNormalizeProviderResponseFillsDefaults：回應正規化預設值測試

當上游 Provider 回傳資料缺少必要欄位時，NormalizeProviderResponse 需補齊標準欄位。本測試驗證：

- 自動注入 provider 欄位
- 若缺失則自動生成 id
- 若缺失則補上 created (Unix timestamp)
- 若缺失則設定 object = "chat.completion"

此測試確保所有回應皆符合系統內部統一의 回應格式。

#### 1. TestNormalizeProviderResponseKeepsExisting：既有欄位保留測試

當 Provider 已提供完整欄位時，系統不應覆寫其值：

- 保留原有的 id、created、object
- 保留其他自訂欄位 (例如 value)
- 僅額外注入 provider 欄位

此測試避免在正規化過程中破壞原始回應語意。

#### 1. TestWriteJSON：成功回應輸出測試



驗證 WriteJSON 函式能正確產生 HTTP JSON 回應：

- 設定正確 HTTP 狀態碼
- 設定 Content-Type: application/json
- 正確序列化並輸出 JSON 內容

#### 1. TestWriteError：錯誤回應輸出測試

驗證 WriteError 函式的錯誤回應格式與 header：

- 設定正確 HTTP 狀態碼
- 設定 Content-Type: application/json
- 回傳標準化錯誤格式：

```
{
  "error": {
    "code": "...",
    "message": "..."
  }
}
```

確保前端與呼叫方能一致解析錯誤資訊。

## 4. AI Collaboration

在本專案中，我們不僅將 AI 視為程式碼生成工具，更將其視為「初級架構師」。我們採用了「規劃優先 (Planning-First)」的協作模式，並引入外部文檔上下文 (Context) 來確保程式碼的正確性。

### 4.1 Prompt Engineering：Planning-First

我們發現直接要求 AI 寫程式碼容易產生邏輯漏洞。因此，我們採用 **"Plan → Review → Implement"** 的三階段 Prompt 策略。

- **關鍵 Prompt 範例 (Schema Mapping 任務)：**

Context: 我需要將 Anthropic 的 API 回應 (system, content 陣列) 轉換為 OpenAI 的標準格式 (choices, message 結構)。

Input: 附上 Anthropic 官方文檔的 JSON Response 範例。

Task:

1. **Analyze:** 請先分析兩者 JSON 結構的差異，列出需要映射的欄位。
2. **Plan:** 不要直接寫程式碼。請先用虛擬碼 (Pseudocode) 或條列式說明轉換邏輯，特別是 `usage` token 的計算方式。

3. **Wait:** 等我確認上述計劃無誤後，再開始撰寫 Go struct 與轉換函數。

- **成效：**透過這個流程，我們在「實作前」就攔截了 2 次欄位型別錯誤，避免了無效的來回 Debug。

## 4.2 Hallucination Fixes and MCP

大型語言模型常因訓練資料過時，給出錯誤的 Go Standard Library 用法或過時的第三方庫版本。為了與「幻覺」對抗，我們採用了 **RAG (Retrieval-Augmented Generation)** 的概念。

- **問題案例：**
  - 在實作 `http.Client` 時，AI 曾建議使用一個已棄用的 `ioutil.ReadAll` 方法，且對於 Anthropic 最新版 API 的 Header 要求 (如 `anthropic-version`) 不清楚。
- **修正方案 (The Fix)：**
  - 我們不依賴 AI 的內部知識，而是利用 **MCP (Model Context Protocol)** 工具 (如 Context7 或類似文檔抓取工具)，讓 LLM 直接讀取最新的 Go 官方文檔與 Anthropic API Reference。
  - **Prompt 調整：**「請不要猜測 API 參數。請根據我透過工具抓取的『Anthropic API 2024-02-29 版本文檔』內容，來生成對應的 Go Struct tags。」
- **結果：**AI 成功生成了包含正確 `json:"content,omitempty"` 標籤的程式碼，且一次編譯通過，完全消除了引用不存在函數的幻覺。

## 4.3 Vibe Coding Insights

- **角色轉變：**開發者的角色從「手寫程式碼 (Coder)」轉變為「程式碼審查者 (Reviewer)」。
- **加速開發：**AI 極其擅長處理繁瑣的 **Boilerplate Code**，例如生成數十行的 Struct 定義與 JSON Tags。這讓我們能將精力集中在最核心的「路由邏輯」與「併發控制」設計上。
- **Planning 的重要性：**我們學到「讓 AI 先思考再行動」是提升程式碼品質的關鍵。透過 Review AI 的計劃，我們不僅修正了 AI，也加深了自己對系統架構的理解，達到了真正的人機協作。
- **善用 MCP 工具：**使用 Context7 MCP 來讓 LLM 找到官方文件得到最新且正確的程式寫法